

AIM2- Preparations

Week 2 — Day 2: Pandas Fundamentals

Today we move from **NumPy arrays** into **Pandas**, which is one of the most important tools for data analysis.

We will focus only on:

- What Pandas is
- Series
- DataFrame
- Creating DataFrames
- Inspecting data
- Selecting rows and columns
- Basic filtering

We will **not** start data cleaning yet.

1. What is Pandas?

Pandas is a Python library used for working with structured data.

Think of data like this:

Name	Math	Python	AI
Krishna	85	92	88
Rahul	75	80	75
Priya	91	95	94
Amit	65	70	68

NumPy can store this as a **2D array**.

But Pandas gives the data:

- column names
- row labels
- easier filtering
- easier analysis
- easier loading from CSV/Excel files

That makes it much more convenient for real datasets.

Import it like this:

```
import pandas as pd
```

pd is the standard alias.

2. NumPy vs Pandas

Your NumPy data looked like:

```
import numpy as np

scores = np.array([
    [85, 92, 88],
    [75, 80, 75],
    [91, 95, 94],
    [65, 70, 68]
])
```

NumPy understands values, but it doesn't naturally know:

Column 0 = Math

Column 1 = Python

Column 2 = AI

With Pandas:

```
import pandas as pd

data = {
    "Math": [85, 75, 91, 65],
    "Python": [92, 80, 95, 70],
    "AI": [88, 75, 94, 68]
}

df = pd.DataFrame(data)

print(df)
```

Output:

	Math	Python	AI
0	85	92	88
1	75	80	75
2	91	95	94
3	65	70	68

Now the columns have names.

3. What is a Series?

A Pandas Series is basically a **single column of data**.

Example:

```
import pandas as pd

scores = pd.Series([85, 75, 91, 65])

print(scores)
```

Output:

```
0    85
1    75
2    91
3    65
dtype: int64
```

A Series has:

- values
- index
- datatype

You can think of it as:

one labeled column

4. Custom Index in Series

You can provide labels:

```
scores = pd.Series(
    [85, 75, 91, 65],
    index=["Krishna", "Rahul", "Priya", "Amit"]
)

print(scores)
```

Output:

```
Krishna    85
Rahul      75
Priya      91
Amit       65
dtype: int64
```

Now:

```
print(scores["Priya"])
```

returns:

91

This is already more descriptive than:

```
scores[2]
```

5. What is a DataFrame?

A DataFrame is a table with:

- rows
- columns
- labels
- possibly different datatypes

You can think of a DataFrame as something similar to:

- Excel worksheet
- SQL table
- Java collection of objects displayed as rows

Example:

```
students = {  
    "Name": ["Krishna", "Rahul", "Priya", "Amit"],  
    "Math": [85, 75, 91, 65],  
    "Python": [92, 80, 95, 70],  
    "AI": [88, 75, 94, 68]  
}
```

```
df = pd.DataFrame(students)
```

```
print(df)
```

Output:

	Name	Math	Python	AI
0	Krishna	85	92	88
1	Rahul	75	80	75
2	Priya	91	95	94
3	Amit	65	70	68

This is the core Pandas structure.

6. type()

Try:

```
print(type(df))
```

You should see something similar to:

```
<class 'pandas.core.frame.DataFrame'>
```

And:

```
print(type(df["Math"]))
```

should show a Pandas Series.

That gives you an important distinction:

```
df          → DataFrame  
df["Math"] → Series
```

7. shape

Just like NumPy:

```
print(df.shape)
```

Output:

```
(4, 4)
```

Meaning:

4 rows

4 columns

Because we have:

Name

Math

Python

AI

8. columns

To see column names:

```
print(df.columns)
```

You may see:

```
Index(['Name', 'Math', 'Python', 'AI'], dtype='object')
```

You can also convert them to a list:

```
print(df.columns.tolist())
```

Output:

```
['Name', 'Math', 'Python', 'AI']
```

9. index

```
print(df.index)
```

You may see:

```
RangeIndex(start=0, stop=4, step=1)
```

That's Pandas' default row index.

10. head()

One of the most important Pandas methods.

```
print(df.head())
```

It displays the first 5 rows by default.

For a large dataset:

```
df.head()
```

is usually one of the first things you run.

You can **specify a number**:

```
print(df.head(2))
```

This shows the first 2 rows.

11. tail()

```
print(df.tail())
```

shows the last 5 rows.

Or:

```
print(df.tail(2))
```

shows the last 2.

12. info()

Another extremely important function.

```
df.info()
```

This gives information about:

- number of rows
- number of columns
- column names
- non-null values
- data types

For real datasets, you'll use this constantly.

13. dtypes

You can inspect just the column datatypes:

```
print(df.dtypes)
```

Example output:

```
Name      object
Math      int64
Python     int64
AI         int64
dtype: object
```

You may wonder why strings say:

object

For now, just understand:

object often represents text/string-like data in Pandas.

We'll go deeper later.

14. Selecting One Column

Given:

```
df = pd.DataFrame(students)
```

You can select:

```
print(df["Math"])
```

Output:

```
0    85
1    75
2    91
3    65
Name: Math, dtype: int64
```

Important:

```
df["Math"]
```

returns a **Series**.

15. Selecting Multiple Columns

You can select multiple columns:

```
print(df[["Name", "Math"]])
```

Notice the double brackets.

```
df["Math"]
```

one column → Series

```
df[["Math", "Python"]]
```

multiple columns → DataFrame

This distinction is very important.

16. Accessing Rows with loc

Pandas provides:

```
.loc[]
```

for label-based selection.

Example:

```
print(df.loc[0])
```

Output:

```
Name      Krishna
Math       85
Python     92
AI         88
Name: 0, dtype: object
```

You can also get one cell:

```
print(df.loc[0, "Python"])
```

Output:

```
92
```

Think:

```
df.loc[row, column]
```

17. Accessing Rows with `iloc`

`iloc` uses numerical positions.

```
print(df.iloc[0])
```

gets the first row.

And:

```
print(df.iloc[0, 2])
```

means:

```
row position 0  
column position 2
```

Output:

92

because column position 2 is Python.

loc vs iloc

Remember:

```
loc → labels/names  
iloc → integer positions
```

Example:

```
df.loc[0, "Python"]
```

uses the column name.

While:

```
df.iloc[0, 2]
```

uses the column position.

18. Filtering

This should feel familiar because of yesterday's NumPy Boolean filtering.

Suppose:

```
print(df["Math"] > 80)
```

Output:

```
0    True
1   False
2    True
3   False
Name: Math, dtype: bool
```

Then:

```
print(df[df["Math"] > 80])
```

returns only matching rows.

Output:

	Name	Math	Python	AI
0	Krishna	85	92	88
2	Priya	91	95	94

This is directly connected to:

```
scores[scores > 70]
```

from NumPy.

19. Multiple Conditions

You can combine conditions:

```
result = df[
    (df["Math"] > 80) &
    (df["Python"] > 90)
]

print(result)
```

Important:

Use:

&

instead of Python's normal:

and

And put each condition in parentheses.

Correct:

`(df["Math"] > 80) & (df["Python"] > 90)`

Not:

`df["Math"] > 80 and df["Python"] > 90`

20. Adding a New Column

This is useful.

You can calculate:

```
df["Average"] = (
    df["Math"] +
    df["Python"] +
```

```
df["AI"]  
) / 3
```

Then:

```
print(df)
```

Now you'll have:

	Name	Math	Python	AI	Average
0	Krishna	85	92	88	88.333333
1	Rahul	75	80	75	76.666667
2	Priya	91	95	94	93.333333
3	Amit	65	70	68	67.666667

This is a major advantage of Pandas.

21. Better Way to Calculate the Average

You can also do:

```
df["Average"] = df[["Math", "Python", "AI"]].mean(axis=1)
```

Notice the NumPy concept appearing again:

`axis=1`

Because we want the mean across columns for every row.

This is why yesterday's NumPy work mattered.

22. Basic Statistical Summary with `describe()`

Try:

```
print(df.describe())
```

Pandas will automatically calculate statistics for numeric columns such as:

- count
- mean
- standard deviation
- minimum
- maximum
- quartiles

You don't need to memorize every value yet.

For now, understand:

```
describe()
```

 gives a quick statistical summary of numerical columns.

Day 2 Practice

Now it's your turn.

Exercise 1 — Create a Series

Create a Pandas Series:

Krishna → 85

Rahul → 75

Priya → 91
Amit → 65

Then print:

- entire Series
- Krishna's score
- Priya's score
- Series datatype
- Series size

Exercise 2 — Create a DataFrame

Create this DataFrame:

Name	Age	City
Krishna	28	London
Rahul	30	Toronto
Priya	27	Ottawa
Amit	32	London

Print:

- full DataFrame
- shape
- columns
- datatypes
- first 2 rows
- last 2 rows

Exercise 3 — Column Selection

Using the same DataFrame:

Print:

1. Only Name
2. Only City
3. Name and Age
4. Name and City

Then tell me:

Which ones return a Series?

Which ones return a DataFrame?

Exercise 4 — loc and iloc

Using the same DataFrame:

Find:

1. Rahul's full row using loc
2. Priya's city using loc
3. First row using iloc
4. Amit's age using iloc

Exercise 5 — Filtering

Create:

```
students = {  
    "Name": ["Krishna", "Rahul", "Priya", "Amit"],  
    "Math": [85, 75, 91, 65],  
    "Python": [92, 80, 95, 70],  
    "AI": [88, 75, 94, 68]  
}
```

Create a DataFrame and filter:

1. Students with Math > 80
2. Students with Python >= 90
3. Students with AI < 80
4. Students with Math > 80 AND Python > 90

Exercise 6 — Add Average Column

Using the student DataFrame:

Create:

Average

using:

```
mean(axis=1)
```

Then print:

Name + Average

Finally filter students with:

Average >= 80



Day 2 Mini-Project

Build:

Student Performance DataFrame Analyzer

Use:

Name	Math	Python	AI
Krishna	85	92	88

Rahul	75	80	75
Priya	91	95	94
Amit	65	70	68
Neha	88	85	90

Your program should:

1. Create a Pandas DataFrame
2. Print the full dataset
3. Print its shape
4. Print column names
5. Print datatypes
6. Display the first 3 rows
7. Display the last 2 rows
8. Add an Average column
9. Display only Name and Average
10. Find students with Average ≥ 80
11. Find students with Math ≥ 85
12. Find students with both Python ≥ 85 and AI ≥ 85
13. Print a statistical summary using `describe()`
14. Print the highest average
15. Print the lowest average